Exercise 4 — Agentic Orchestration

> **Goal:** add a bounded AI agent that proposes an alpaca event plan. The agent uses OpenAI via the **AI Agent Sub-process**, has two tools at its disposal, and hands its proposal to the Chief Alpaca Officer for the final call.

What you will learn

- How the **AI Agent Sub-process** maps to BPMN (ad-hoc sub-process with the `aiagent-job-worker:1` task type).
- How to keep deterministic control in BPMN while letting the LLM reason inside one bounded step.
- How to define tools using `fromAi(...)` parameters and the `toolCallResult` convention.
- How to wire the OpenAI provider — model, temperature, system prompt, memory and call limits.

Prerequisites

1. Exercises 1, 2 complete (IDP, workers, listeners).
2. The `OPENAI_API_KEY` secret already configured in the SaaS Connector secrets (Exercise 3, Step 1).

Step 1 — Run the agent as is

1. Open the BPMN in Web Modeler and start a Play session.
2. Walk to the agent ad-hoc sub-process and let the LLM call the tools.
3. Inspect `agent.responseText` — you should see structured JSON like:

```
{
  "proposedSchedule": [
    { "time": "10:00", "activity": "Lobby meet-and-greet — soft-hoof
```

```
mats only." },
    { "time": "13:30", "activity": "Floor tour 2 — visit 3 squads." }
  ],
  "riskNotes": "Avoid elevators per policy. Past visit notes show
carpet sensitivity on level 3.",
  "recommendedDecision": "APPROVE"
}
```

Step 2 — Add a new tool

Add a UserTask so that the agent can ask questions or provide important information (this could potentially also an Email task or any other kind of conversation channel) Provide the following configuration:

1. Name: `Request or provide additional information`
2. ID: `tool_AdditionalInformation`
3. Element documentation: `Use this tool if you have any questions or you need more information about the planned alpaca office visit or you need confirmation that certain requirements are met.`
4. InputMapping:
    - Local Variable Name: `alpacaRequestContext`
    - Variable Assignment Value: `=fromAi(toolCall.alpacaRequestContext, "Summarize the available information about the planned alpaca office visit", "string")`
    - Local Variable Name: `agentQuestion`
    - Variable Assignment Value: `=fromAi(toolCall.aiAsk, "What is your question to the applicant?", "string")`
5. OutputMapping:
    - Process Variable Name: `toolCallResult`
    - Variable Assignment Value: `={answer: applicantResponse}`
6. run Step 1 again, the behavior should not change.

7. Now change the `notes` attribute of the `Get past visit history` task to `"Minor carpet incident on level 3. For future events an \"Alpaca Incident Cleanser\" must be provided!"`

8. Run again an instance. The AI should trigger a UserTask since it became aware of new requirements.

Three things to memorise:

1. **Tool name** = the activity `name` attribute. The LLM sees `Request or provide additional information`.
2. **Tool description** = the `<bpmn:documentation>` element. This is what teaches the LLM *when* to call the tool. Be specific.
3. **Tool parameters** = anything you wrap in `fromAi(toolCall.<param>, "...", "<type>")`. The connector auto-generates a JSON-Schema tool definition for the LLM. The `toolCall.` prefix is mandatory.

The **result** must land in a variable named `toolCallResult`. The ad-hoc `outputCollection` collects every tool result and feeds it back to the LLM with the matching `toolCall._meta.id`. Without this convention, the agent stalls after the first tool call.

Observe in Operate:

- The agent ad-hoc sub-process activates the LLM job (`io.camunda.agenticai:aiagent-job-worker:1`).
- The variable `agent.context` accumulates the conversation; `agent.toolCalls` shows tool invocations.
- When the agent decides it has enough information, it stops the loop and writes `agent.response`.

Step 3 — Discuss the guardrails

The exercise is also a chance to discuss *what the agent is not allowed to do*. Verify by inspection:

Check	Where enforced
Budget over €300 → reject	DMN `budget-approval.dmn` (deterministic).
Vet certificate expired/missing → loop	DMN `document-validity.dmn`.

Check	Where enforced
HR / Facilities veto	Human user tasks before the agent runs.
Final approval	**Chief Alpaca Officer** user task after agent.

The agent *cannot* skip any of these. Even a perfect plan gets thrown away if the approval steps say no — that's the whole point of bounded agentic orchestration.

Acceptance criteria

- [ ] The agent executes the ad-hoc sub-process and produces a structured `agent.response` JSON.
- [ ] At least one tool call appears in `agent.toolCalls`.
- [ ] The CAO task surfaces the agent's plan via the form template.
- [ ] An approved CAO decision leads to `EndEvent_Confirmed`; a rejected decision leads to `EndEvent_RejectedCAO`.

Reference links

- AI Agent Sub-process
- AI Agent connector reference
- `fromAi` tool parameter helper
- Test AI agents with CPT
- Camunda secrets (SaaS)